Function Prototypes and Function Calls

Understanding the fundamentals of C programming functions

Function Prototype

Definition

A function prototype is a declaration of a function that tells the compiler:

Function's Name

What the function is called

Return Type

What type of value it will return

Parameters

What arguments it will take

It's written before the main() function, so that the compiler knows about the function before it's used.

Function Prototype 🧠

Why We Need It

Without a prototype, the compiler doesn't know:

Number of Arguments
How many arguments the function needs

Data Types
What data types those arguments are

Return Value
What type of value (if any) it returns

This can lead to **errors** or **unexpected behavior** in your program.

Function Prototype ✅

Syntax

```
return_type function_name(parameter_type1, parameter_type2, ...);
```

Example

```
#include <stdio.h>

// Function prototype (declaration)
int add(int, int);

int main() {
    int result = add(5, 10); // function call
    printf("Sum = %d", result);
    return 0;
}

// Function definition
int add(int x, int y) {
    return x + y;
}
```

Function Prototype

Explanation

```
int add(int, int);
```

→ tells the compiler that a function named `add` exists, it takes two integers and returns an integer.

Compiler Benefits

Later, when we define add(), the compiler already knows how to handle the call in main().

Type Checking

The compiler can verify correct types are used

Parameter Count

The compiler ensures the correct number of arguments

Call by Value

Definition

In call by value, a **copy** of the actual argument is passed to the function.

How it Works

The function works on this copy, not the original variable. Changes made inside the function do NOT affect the original variable.

Copies Data
Creates a duplicate of the original value

Original Unchanged
Original variable remains the same

Memory Usage
Uses more memory due to copying

Call by Value ✅

Example

```c
#include <stdio.h>

void changeValue(int x) {
    x = 50; // changes only local copy
}

int main() {
    int a = 10;
    printf("Before function call: a = %d\n", a);
    changeValue(a);
    printf("After function call: a = %d\n", a);
    return 0;
}
```

Call by Value 📄

Output

```
Before function call: a = 10
After function call: a = 10
```

Explanation

📋 Copying Process

a is copied into x.

✏️ Local Changes

The function changes the copy, not the real a.

Original Unchanged

The original a remains unchanged.

Call by Reference

Definition

In call by reference, instead of passing values, we pass the **addresses** (memory locations) of the variables.

Direct Access

That means the function can directly modify the original variables, because it has access to their memory addresses.

Passes Address
Sends memory location instead of value

Original Modified
Changes affect the original variable

Memory Efficient
Uses less memory than call by value

Call by Reference ✅

Example

```c
#include <stdio.h>

void changeValue(int *x) {
    *x = 50; // modifies the actual variable through pointer
}

int main() {
    int a = 10;
    printf("Before function call: a = %d\n", a);
```

```
    changeValue(&a); // pass address of a
    printf("After function call: a = %d\n", a);
    return 0;
}
```


Call by Reference 📜

Output

```
Before function call: a = 10
After function call: a = 50
```

Explanation

📍 Passing Address

&a passes the address of a to the function.

Dereferencing

Inside changeValue(), *x means "the value at the address of a".

Direct Modification

When we change *x, we are actually changing a in memory.

Summary Table

Feature	Call by Value	Call by Reference
What is passed	Value (copy)	Address (reference)

Feature	Call by Value	Call by Reference
Function works on	Copy of variable	Actual variable
Changes reflect in caller?	❌ No	✅ Yes
Example syntax	swap(a, b);	swap(&a, &b);
Function parameters	Normal variables	Pointers

Quick Real-Life Analogy 🎯

Call by Value

You lend someone a photocopy of your document. They can write on it, but your original

Call by Reference

You lend someone your original document. Any change they make affects your copy directly.

stays the same.

Photocopy

Changes only affect the copy

Original Document

Changes affect the original

Understanding these concepts is crucial for efficient C programming! 🚀